

Dynamic Memory Allocation

元培数科 王恩博

Let's get started →

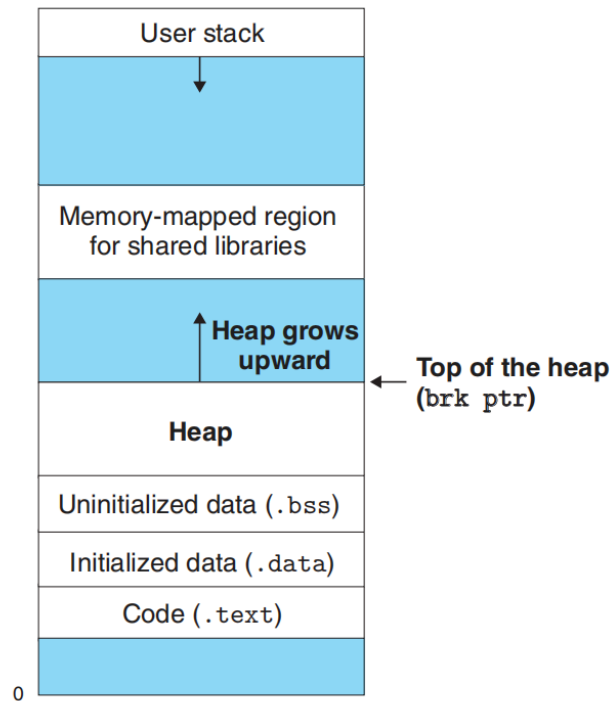
2025/12/10



Knowledge Review

基本概念

- `brk` 指针指向堆顶
- 显示分配器：显示释放已分配的块
- 隐式分配器（垃圾收集器）：自动检测块不再被使用，进而自动释放
- `void *malloc(size_t size);` 分配一个块，并且 (8/16) 对齐
- `calloc` 分配块并且初始化为0
- `void *sbrk(intptr_t incr);` 扩展或者收缩堆，成功返回 `brk` 旧值，出错返回-1。
- `void free(void *ptr);` 不返回内容。如果 `ptr` 不是 `malloc/calloc/realloc` 分配块的起始位置，行为未定义。



malloc 与 free 示例



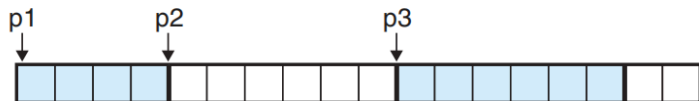
(a) `p1 = malloc(4*sizeof(int))`



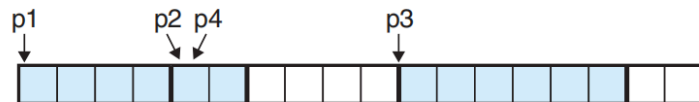
(b) `p2 = malloc(5*sizeof(int))`



(c) `p3 = malloc(6*sizeof(int))`



(d) `free(p2)`



(e) `p4 = malloc(2*sizeof(int))`

为什么动态分配

- 例：做OJ题时候数组开多大？
- 浪费 or 不够用
- 不可预知/不同次差距极大

分配器要求和目标

- 处理任意请求序列
- 立即响应请求
- 只使用堆
- 对齐要求
- 不修改已分配的块

评价指标：

- 吞吐率：最差分配时间 $O(n)$,释放时间 $O(1)$ (n 为空闲块数量)
- 内存利用率
- **峰值利用率：**

$$U_k = \frac{\max_{i \leq k} P_i}{H_k} \quad P_k \text{ 为有效载荷之和, } H_k \text{ 为堆大小 (单调递增)}$$

使得 U_{n-1} 最大化

寻找吞吐率和利用率的平衡。

碎片

- 内部碎片：为了**满足最小大小**/对齐带来的碎片
- 外部碎片：所有空闲块加在一起够，但单个空闲块不够（直观理解：空闲块被分碎了）

为什么会设置最小块大小？

- 可能增加了内部碎片，但会减少外部碎片的可能，不把空闲块分的太碎

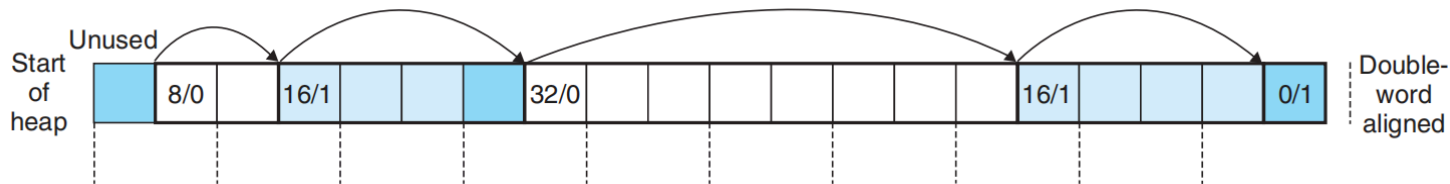
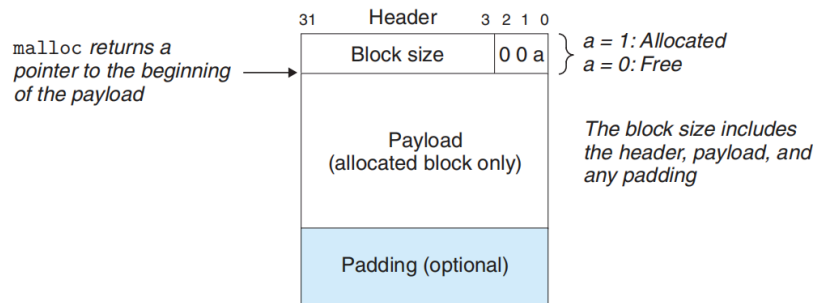
实现

- 组织：如何记录空闲块？
隐式空闲链表、显式空闲链表、分离空闲链表
- 放置：选哪个空闲块？
首次适配、下一次适配、最佳适配
- 分割：拿走需要的剩下的怎么办？
整个使用、分割
- 合并：怎么处理释放的块？
立即合并、推迟合并

隐式空闲链表

Implicit Free Lists

- size都包括什么?
- 因为大小8字节对齐, 头部最后三位不需要, 可用作标记
- padding 用于对齐/分配器的feature, 可用于消除外部碎片
- 分配块的时候需要从头搜索



放置与分割

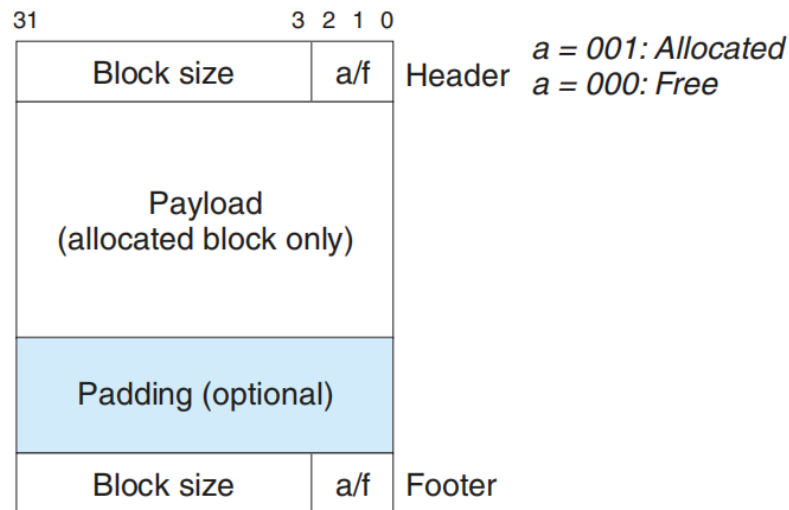
- 首次适配：从头开始搜索
- 下一次适配：从上一次查询结束位置开始搜索
- 最佳适配：满足条件的最小的
- **内存利用率（平均）：最佳适配 > 首次适配 > 下一次适配**
- 找不到怎么办？合并空闲块/请求更多堆空间
- 不分割整个使用：内部碎片
- 分割：可能导致外部碎片
- 放置策略与分割策略需要匹配权衡。**这需要你自己衡量**

合并空闲块

- 假碎片：相邻的小空闲块
- 立即合并：释放的时候就合并
- 推迟合并：稍晚的时候（设定时间间隔/一直等到某次分配找不到符合的空闲块）再合并

带边界标记的合并

- 关键问题：怎么找到前面的空闲块？
- 单向链表只能从头搜索，不是 $O(1)$
- 解决：增加一个脚部
- 新问题：浪费空间
- 只有空闲块需要脚部，已分配的块不需要
- 怎么判断前面的块空不空闲？
- 把这个状态也放到当前块头部低三位中



带边界标记实现实例

m1	a
m1	a
n	a
n	a
m2	a
m2	a



m1	a
m1	a
n	f
n	f
m2	a
m2	a

情况 1

m1	a
m1	a
n	a
n	a
m2	f
m2	f



m1	a
m1	a
n+m2	f
n+m2	f

情况 2

m1	f
m1	f
n	a
n	a
m2	a
m2	a



n+m1	f
n+m1	f
m2	a
m2	a

情况 3

m1	f
m1	f
n	a
n	a
m2	f
m2	f

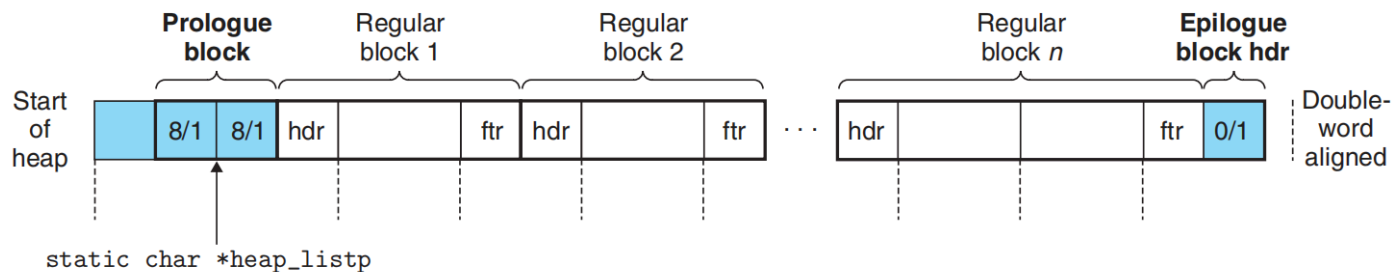


n+m1+m2	f
n+m1+m2	f

情况 4

隐式空闲链表的固定格式

- 序言块：8字节已分配块
- 结尾块：大小为0的已分配块



代码实现

- 可参考大班课件或教材，做了lab应该会深刻理解
- 主要的新知识是理解宏定义

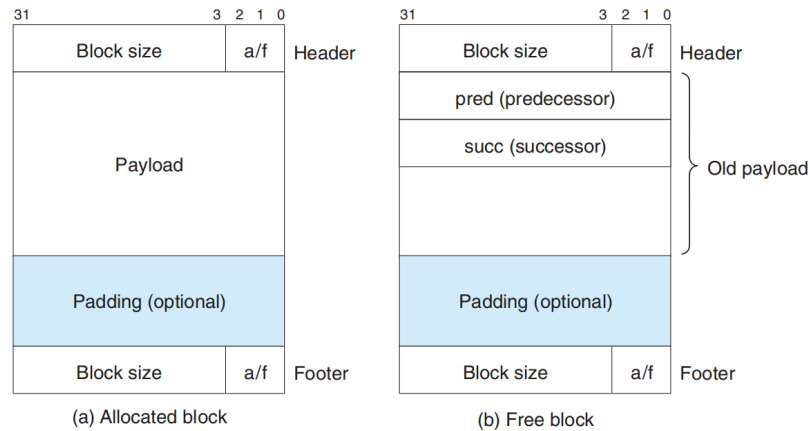
```
#define WSIZE      4      /* Word and header/footer size (bytes) */ //line:vm:mm:beginconst
#define DSIZ      8      /* Double word size (bytes) */
#define CHUNKSIZE (1<<12) /* Extend heap by this amount (bytes) */ //line:vm:mm:endconst

#define MAX(x, y) ((x) > (y)? (x) : (y))
/* Pack a size and allocated bit into a word */
#define PACK(size, alloc) ((size) | (alloc)) //line:vm:mm:pack
/* Read and write a word at address p */
#define GET(p)      (*(unsigned int *)(p))      //line:vm:mm:get
#define PUT(p, val) (*(unsigned int *)(p) = (val)) //line:vm:mm:put
/* Read the size and allocated fields from address p */
#define GET_SIZE(p) (GET(p) & ~0x7)             //line:vm:mm:getsize
#define GET_ALLOC(p) (GET(p) & 0x1)             //line:vm:mm:getalloc
/* Given block ptr bp, compute address of its header and footer */
#define HDRP(bp)    ((char *) (bp) - WSIZE)      //line:vm:mm:hdp
#define FTRP(bp)    ((char *) (bp) + GET_SIZE(HDRP(bp)) - DSIZ) //line:vm:mm:frp
/* Given block ptr bp, compute address of next and previous blocks */
#define NEXT_BLK(p) ((char *) (bp) + GET_SIZE(((char *) (bp) - WSIZE))) //line:vm:mm:nextblkp
#define PREV_BLK(p) ((char *) (bp) - GET_SIZE(((char *) (bp) - DSIZ))) //line:vm:mm:prevblkp
```

显式空闲链表

Explicit Free Lists

- 减少放置时间: $O(n)$, n 从所有块数目减少到空闲块数目
- 释放时间取决于链表组织方式
- 释放的放到链表最前面(LIFO): 释放时间 $O(1)$
- 按地址排序: 释放时间 $O(n)$, 取决于前驱数目, 不过可以提高在首次适配情况下的内存利用率
- 问题: 提高了最小块大小 (得放得下两个指针+首部+脚部), 内部碎片加剧



分离的空闲链表

Segregated Free Lists

- 思想：分离存储，按块大小分为等价类（大小类）
- 每个大小类一个空闲链表
- 减少放置时间：在相应的空闲链表中搜索，找不到的时候去**更大的**大小类的链表中搜索

简单分离存储

Simple Segregated Storage

- 每个大小类只包括大小相同的块（取最大值）
- 不分割、不合并
- 分配和释放时候速度很快
- 不需要头部、脚部，只需要单向链表，最小块只需一个字
- 容易造成内部碎片和外部碎片
- 例：先对第一个大小类做大量分配和释放，然后对第二个大小类做大量分配和释放，以此类推.....
- 因为前面释放的小块不会合并，会造成大量外部碎片

分离适配

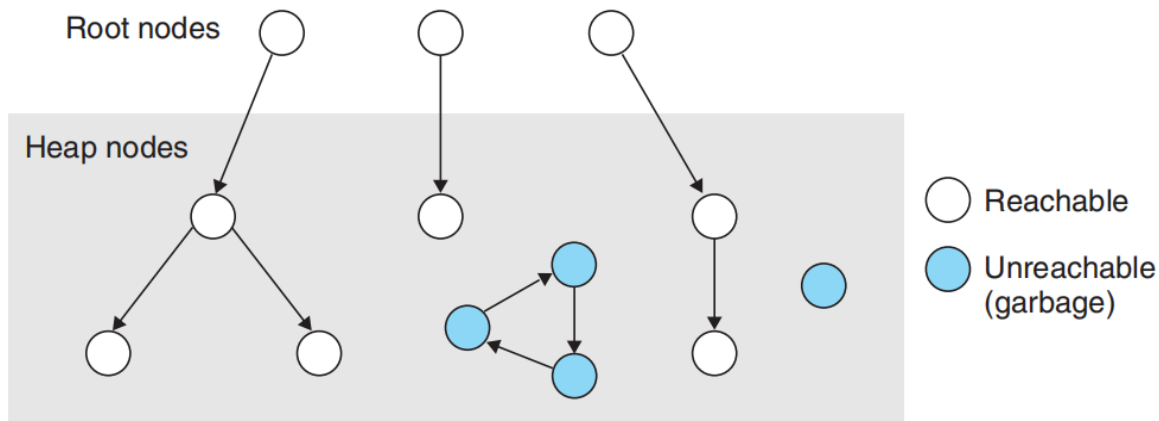
Segregated Fits

- 对相应的空闲链表**首次适配**
- 可以做分割，将剩余的插入到**适当的空闲链表**中
- 新请求的堆空间也进行分割，将剩余部分插入适当空闲链表
- 释放的时候进行合并
- 对分离空闲链表的首次适配近似于对整个堆的最佳适配
- **伙伴系统** (Buddy Systems) : 分离适配的特例
- 每个块大小是2的幂，主要方便合并
- 合并时：为保证合并后大小还是2的幂，跟块 A 合并的找A的伙伴，只有一位地址不同
- 例：A地址为xxx10000，大小为32字节，其伙伴是xxx00000
- S -> A|B -> C|D|E|F
- 释放了D 和E 此时不会合并

垃圾收集

Garbage Collection

- 垃圾：程序不再需要的已分配块
- 根节点：不在堆中的位置，包含指向堆中的指针。**寄存器、栈中变量、全局变量**
- 可达性：有从根节点出发到达p的路径
- C 中无法判断一个变量是不是指针，因此使用**保守的垃圾收集器**
- 可以在无法分配的时候进行垃圾收集



Mark&Sweep 垃圾收集器

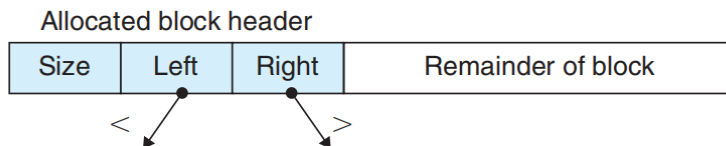
(a) mark function

```
void mark(ptr p) {
    if ((b = isPtr(p)) == NULL)
        return;
    if (blockMarked(b))
        return;
    markBlock(b);
    len = length(b);
    for (i=0; i < len; i++)
        mark(b[i]);
    return;
}
```

(b) sweep function

```
void sweep(ptr b, ptr end) {
    while (b < end) {
        if (blockMarked(b))
            unmarkBlock(b);
        else if (blockAllocated(b))
            free(b);
        b = nextBlock(b);
    }
    return;
}
```

- C 中无法判断一个变量是不是指针，只能全当指针
- 无法判断位置是否指向一个已分配块的有效载荷
- 利用平衡二叉树，进行二分查找



补充

其他的垃圾清扫方法

引用计数

- 对象分配/参数传递：引用计数设为1
- 引用赋值： $u=v$ ： u 指向的对象引用减1、 v 指向的对象引用加1
- 过程返回：局部变量指向对象的引用计数减1
- 如果一个对象的引用计数为0，在删除对象之前，此对象中各个指针所指对象的引用计数减1

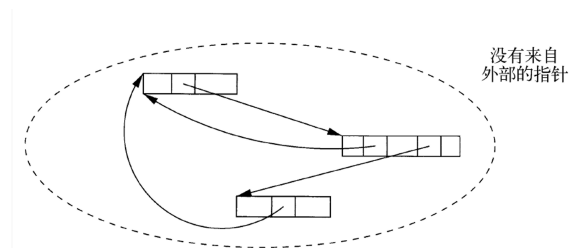
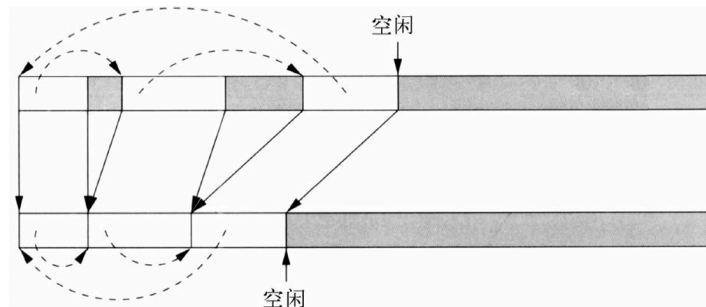


图 7-18 一个不可达的循环数据结构

标记压缩



复制回收

分代回收

.....

实际应用：多种相结合

与内存有关的错误

Common Memory-Related Bugs in C Programs

- 间接引用坏指针 (`scanf("%d", val)`)
- 读未初始化内存
- 允许栈缓冲区溢出 (attacklab)
- 假设指针和它们指向的对象是相同大小的
- 造成错位错误 (数组越界)
- **引用指针，而不是它所指向的对象** (`*size--` 改变了指针的值)
- 误解指针运算 (指针加1不一定是一字节)
- 引用不存在的变量 (栈中局部变量返回之后就不能通过地址引用了)
- 引用空闲堆块中的数据 (已经 free)
- 引起内存泄漏

Homework Review

9.15

9.15 确定下面的 malloc 请求序列得到的块大小和头部值。假设：1)分配器保持双字对齐，使用隐式空闲链表，以及图 9-35 中的块格式。2)块大小向上舍入为最接近的 8 字节的倍数。

请求	块大小（十进制字节）	块头部（十六进制）
malloc(3)		
malloc(11)		
malloc(20)		
malloc(21)		

块大小

块头部

8 (1000)

(1001) 0x9

16 (10000)

(10001) 0x11

24 (11000)

(11001) 0x19

32 (100000)

(100001) 0x21

9.16

- * 9.16 确定下面对齐要求和块格式的每个组合的最小块大小。假设：显式空闲链表、每个空闲块中有四字的 pred 和 succ 指针、不允许有效载荷的大小为零，并且头部和脚部存放在一个四字节的字中。

对齐要求	已分配块	空闲块	最小块大小 (字节)
单字	头部和脚部	头部和脚部	
单字	头部, 但是没有脚部	头部和脚部	
双字	头部和脚部	头部和脚部	
双字	头部, 但是没有脚部	头部和脚部	

已分配块最小大小

空闲块最小大小

块最小大小

$$4+4+1 = 9 \rightarrow 12$$

$$4+2*4+4 = 16$$

16

$$4+1 = 5 \rightarrow 8$$

$$4+2*4+4 = 16$$

16

$$4+4+1 = 9 \rightarrow 16$$

$$4+2*4+4 = 16$$

16

$$4+1 = 5 \rightarrow 8$$

$$4+2*4+4 = 16$$

16

9.19 - 1

* 9.19 下面给出了三组关于内存管理和垃圾收集的陈述。在每一组中，只有一句陈述是正确的。你的任务就是判断哪一句是正确的。

- 1) a) 在一个伙伴系统中，最高可达 50% 的空间可以因为内部碎片而被浪费了。
- b) 首次适配内存分配算法比最佳适配算法要慢一些(平均而言)。
- c) 只有当空闲链表按照内存地址递增排序时，使用边界标记来回收才会快速。
- d) 伙伴系统只会有内部碎片，而不会有外部碎片。

■ 答案：a

a: $\lim_{n \rightarrow \infty} \frac{2^{n-1}-1}{2^n} = \frac{1}{2}$

b: 首次适配平均更快

c: 与空闲链表排序方式无关，目的方便找到内存中相邻块，空闲链表排序方式主要影响分配

d: 前面有举例

■ S -> A|B -> C|D|E|F

■ 释放了D 和E 此时不会合并

9.19 - 2

- 2) a) 在按照块大小递减顺序排序的空闲链表上，使用首次适配算法会导致分配性能很低，但是可以避免外部碎片。
- b) 对于最佳适配方法，空闲块链表应该按照内存地址的递增顺序排序。
- c) 最佳适配方法选择与请求段匹配的最大的空闲块。
- d) 在按照块大小递增的顺序排序的空闲链表上，使用首次适配算法与使用最佳适配算法等价。

■ 答案：d

a: 每次都会切割最大的块，会造成外部碎片

b、c: 可以按照大小递增排序（不必须），找到最小的满足要求的块

d: 首次适配找到的也是最小的大于等于要求的块

9.19 - 3

3) Mark&Sweep 垃圾收集器在下列哪种情况下叫做保守的：

- a) 它们只有在内存请求不能被满足时才合并被释放的内存。
- b) 它们把一切看起来像指针的东西都当做指针。
- c) 它们只在内存用尽时，才执行垃圾收集。
- d) 它们不释放形成循环链表的内存块。

■ 答案：b

Exercises

E1

19. 垃圾收集器的根节点不包括以下哪个节里的数据:

- A. text
- B. data
- C. bss
- D. stack

E1

19. 垃圾收集器的根节点不包括以下哪个节里的数据:

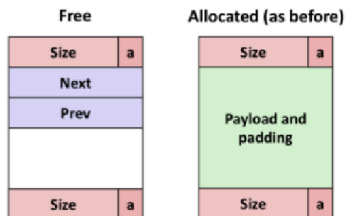
- A. text
- B. data
- C. bss
- D. stack

19. A。其他段中都可能含有指针，其引用的内存需要管理。代码段中则没有这种问题。

E2

12. 现在有一个用户程序执行了如下调用序列

```
void *p1 = malloc(16);
void *p2 = malloc(32);
void *p3 = malloc(32);
void *p4 = malloc(48);
free(p2);
void *p5 = malloc(4);
free(p3);
void *p6 = malloc(56);
void *p7 = malloc(10);
```



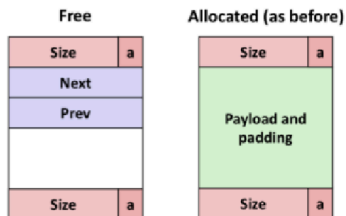
内存分配器内部使用**显式空闲链表**实现，按照地址从低到高顺序来维护空闲链表节点顺序。分配块格式参照上图。每个分配块 16 字节对齐，头部和脚部的大小都是 4 字节。分配算法采用**首次适配**算法，将适配到的空闲块的第一部分作为分配块，剩余部分变成新的空闲块，并采用**立即合并**策略。假设初始空闲块的大小是 1KB。那么以上调用序列完成后，分配器管理的这 1KB 内存区域中，**内部碎片**的总大小是____，链表里**第一个**空闲块的大小是____。

- A. 58Byte, 848Byte
- B. 26Byte, 32Byte
- C. 74Byte, 48Byte
- D. 74Byte, 16Byte

E2

12. 现在有一个用户程序执行了如下调用序列

```
void *p1 = malloc(16);
void *p2 = malloc(32);
void *p3 = malloc(32);
void *p4 = malloc(48);
free(p2);
void *p5 = malloc(4);
free(p3);
void *p6 = malloc(56);
void *p7 = malloc(10);
```



内存分配器内部使用**显式空闲链表**实现，按照地址从低到高顺序来维护空闲链表节点顺序。分配块格式参照上图。每个分配块 16 字节对齐，头部和脚部的大小都是 4 字节。分配算法采用**首次适配**算法，将适配到的空闲块的第一部分作为分配块，剩余部分变成新的空闲块，并采用**立即合并**策略。假设初始空闲块的大小是 1KB。那么以上调用序列完成后，分配器管理的这 1KB 内存区域中，**内部碎片**的总大小是____，链表里**第一个**空闲块的大小是____。

- A. 58Byte, 848Byte
- B. 26Byte, 32Byte
- C. 74Byte, 48Byte
- D. 74Byte, 16Byte

12. D。直接模拟就可以。

E11

第五题。请结合教材第九章“虚拟内存”的有关知识回答问题(15分)

IA32 体系采用小端法、32 位虚拟地址和两级页表。两级页表大小相同，页大小都是 $4\text{ KB} = 2^{12}\text{ Byte}$ ，结构也相同。TLB 采用直接映射，4 位组索引。TLB 和页表每一项格式如图所示：

31	12	11	9	8	7	6	5	4	3	2	1	0
Address of 4KB page frame		Ignored		G	P	D	A	P	U	R		P
					A			C	/	/		
					T			D	T	S	W	

部分位的含义如下：

0 (P)：1 表示存在，0 表示不存在

1 (R/W)：1 表示可写，0 表示只读

2 (U/S)：1 表示内核模式，0 表示用户模式

当系统运行到某一时刻，TLB 有效位为 1 的条目如下（未列出部分都是无效的）：

索引（十进制）	TLB 标记	内容
0	0x0400	0x0ec91313
3	0x02ff	0xd5d2bac01
5	0xd551	0x019fa42d
11	0x55a6	0xfdd3c66b
13	0x5515	0xb591926b

一级页表基址为 $0x00e66000$ ，物理内存中的部分内容如下（均为十六进制）：

地址	内容	地址	内容	地址	内容
00615000	21	00615001	2d	00615002	ee
00615003	c0	006154d0	ff	006154d1	a0
00e66001	a1	00e66002	a4	00e66003	67
00e66004	21	00e66005	57	00e66006	61
00e66d07	00	2167e000	42	2167e001	67
2167e002	9a	2167e003	7c	c0ee2000	6f
c0ee2001	d5	c0ee2002	7e	c0ee2003	48
c0ee24d1	83	c0ee24d2	ec	c0ee2d21	11
c0ee2d22	6b	c0ee2d23	82	c0ee2d24	8a

1. 将 cache 清空，访问一个在主机中的虚拟地址，TLB 命中，没有触发缺页异常，这一过程中，需要访问物理内存____次（3 分）。具体来说，如果该地址为 $y = 0xd5515213$ ，y 地址所具有的实质权限是____（多选题，选对才得分 2 分）。

①可写 ②只读 ③用户模式权限 ④内核模式权限

2. 不考虑第一小问，将 cache 清空，访问一个在主机中的虚拟地址，TLB 不命中，没有触发缺页异常，这一过程中，需要访问物理内存____次（3 分）。具体来说，如果该地址为 $x = 0x004004d0$ ，那么 x 对应的二级页表起始地址是____（填写 16 进制，例如 $0x0123000$ ，2 分），x 地址上单字节的內容是____（填写 16 进制，例如 $0x00$ ，1 分）。

3. 考虑下面计算矩阵和向量乘法代码：

```
1 int *mat_vec_mul(int **A, int *x, int n)
2 {
3     int i, j;
4     int *y = (int *)malloc(n * sizeof(int));
5     for (i = 0; i < n; i++)
6         for (j = 0; j < n; j++)
7             y[i] += A[i][j] * x[j];
8     return y;
9 }
```

① 在 64 位 Linux 机器中运行该代码，输入同一组合法的参数后，每次运行返回的向量的值都不一样，修复这一错误有一种简单的方法，将第____行改为____。（第二空填写 C 代码，每空 1 分，共 2 分）

可能用到的函数：

```
void *memcpy(void *dest, const void *src, size_t n);
void *memset(void *s, int c, size_t n);
void *calloc(size_t nelem, size_t elemsize);
void *realloc(void *ptr, size_t size);
```

② 在进行前一问的测试之前，还在 64 位 Linux 机器上进行过如下用户代码测试（输入 mat_vec_mul 的参数都非零）：

```
int *y = mat_vec_mul(A, x, n);
int z = y[0];
```

结果发生了段错误。通过 gdb 调试发现 mat_vec_mul 函数内并没有发生段错误，但是在初始化变量 z 时发生了段错误，后来发现是参数的输入有问题。写出出现这种错误的充分必要条件____（1 分）

4. Double fault: Intel 处理器中有一种特殊的异常，被称为 double fault。此异常发生表明调用某个故障 (fault) A 的处理程序后又触发了另一个故障 B。正常情况下，故障 B 会有相应异常处理程序来处理，因此两个故障 B 和 A 可以被顺序解决。但是如果处理器无法正常处理故障 B，或是处理了之后依然无法处理故障 A，就会产生 double fault，并终止 (abort)。假设除了缺页异常处理程序外，其他异常处理程序都不会产生新的故障。如果在某次缺页故障时产生了 double fault，其原因可能是____（不定项选择，都选对才得分，1 分）

- ① 运行缺页故障处理程序时，CPU 上的权限位是内核态，但所执行代码段 U/S=0
- ② 运行缺页故障处理程序时，CPU 接收到了键盘发送的 Ctrl + C 信号
- ③ 缺页故障处理程序没有加载到主存中

E11

答案: ←

1. 1 ②④ ←

2. 3 0x00615000 0x48 ←

3. (1) 4 `int *y = (int *)calloc(n, sizeof(int));` 或 ←

`int *y = (int *)Calloc(n, sizeof(int));` ←

其中 `calloc/Calloc` 的两个参数只要乘起来等于 `n * sizeof(int)` 即算对 ←

(2) `n < 0` (或其他等价表达) ←

4. ③ ←

←

←

解析: ←

1. 考察 TLB 的位划分与表项含义, 属于简单题。←

题目告知 TLB 命中且无缺页, 因此只需要访问物理页, 即访问一次物理内存 ←

根据 `y` 的值可以得到 `TLBT=0xd551`, `TLBI=5`, 查表知 `y` 在 TLB 条目的内容为 `0x019fa42d`, `R/W=0`, `U/S=1`, 因此这一页的实质权限为只读、内核模式。←

参见课本第 9.6 和 9.7 节 (P567-582) ←

←

2. 考察虚拟内存地址翻译, 属于简单+中等题。←

题目告知 TLB 未命中且无缺页, 因此需要访问页目录、二级页表和物理页, 共访问三次物理内存。←

计算得 `x` 的 `VPN1=1`, `VPN2=0`, `PPO=0x4d0`。页目录基地址在 `0x00e66000`, 因此相应页目录中所在条目地址为 `0x00e66000+1*4=0x00e66004`, 从表中按小端法读出 `0x00615721`, 因此二级页表存在, 首地址为 `0x00615000`, 这恰好也是相应二级页表中所在条目的地址, 从表中按小端法读出 `0xc0ee2d21`, 因此物理页存在, 首地址为 `0xc0ee2000`, 因此 `x` 的物理地址为 `0xc0ee2000+0x4d0=0xc0ee24d0`。从表中读出一个字节, 为 `0x48`。←

参见课本第 9.6 和 9.7 节 (P567-582) ←

3. 本题考查内存安全。←

(1) 考察 `malloc` 函数的性质, 属于中等题。←

`malloc` 函数并不会清零所分配内存区域, 因此同样的输入可能会有不同的输出, 但 `mat_vec_mul` 函数别的行都不可能产生这一效果, 因此只需要将第 4 行换为 ←

`int *y = (int *)Calloc(n, sizeof(int));` ←

`Calloc` 是一个带错误检查的、会将所分配区域置零的函数。←

参见课本第 9.11.2 节正文例子 (P610)。←

←

(2) 内存安全的综合考察, 属于难题。←

假设 `malloc` 函数分配成功, 那么 `y` 不是空指针。`y[0]` 不可能在外部访问出现段错误, 因此这种情况不成立, `malloc` 一定出现了问题, 导致 `y` 是空指针。但是此时 `mat_vec_mul` 并没有触发段错误, 说明函数内部一次对 `y` 的访问都没有, 这只有可能循环都没有进入, 因此必须有 `n<=0`, 根据题设, 参数是非零的, 所以 `n<0`。以上推导出了 `n<0` 是必要条件。←
下面推导 `n<0` 是充分条件, 即任何 `n<0` 都会导致这样的错误。为此, 只需要说明 `malloc` 函数一定分配失败。首先, 这是 Linux 64 位系统, 因此地址空间为 64 位, `size_t` 为 64 位, 用户虚拟地址空间为 48 位。注意 `n*sizeof(int)` 会将 `n` 的类型转为 `long`, 再转为 `unsigned long`, 而 `n` 只有 32 位, 所以符号扩展之后最高位至少有 33 个 1, 乘 `sizeof(int)`, 即 4 之后, 最高位还有至少 29 个 1, 因此 `n*sizeof(int)>263>248`, 超过了用户虚拟地址空间的大小, 因此无论如何, `malloc` 函数都会分配失败。←

参见课本第 9.8 节 (P576)、9.9.1 节 (P588)、2.2.8 节 (P58-59)。←

←

4. 本题考查对缺页故障、异常处理的综合理解, 属于难题。←

① 如果 CPU 权限位是内核态, 它自然可以运行用户级代码, 不会触发异常。←

② 如果缺页故障处理程序运行时, 收到外部中断, 这不属于故障, 因此不会触发。←

③ 如果缺页故障处理程序不在主存, 那么调用它会触发缺页故障, 于是这个故障又会导致调用缺页故障处理程序, 但它仍然不在主存, 故这一故障无法被解决, 于是触发 `double fault`。←
相关内容参见课本第 8.1 节 (P502-507)、9.7.2 节 (P581-582) 和 Intel 64 and IA-32 Architectures Software Developer's Manual - Volume 3: System Programming Guide 的第 6.15 节。←

Notices

Lab 测验

THANKS

Made by WEB-05

webrun@stu.pku.edu.cn

Reference: [Arthals]'s template.



扫一扫上面的二维码图案，加我为朋友。